

Big Data Analytics Pipeline Using Apache Spark, Hadoop HDFS, Apache Hive and Apache Superset Phase 1

1. Introduction

E-commerce platforms generate transactional data at a scale that quickly outgrows what a single machine or a traditional relational database can comfortably process. Turning that raw data into something a business can actually act on sales trends, customer distribution, order volume over time requires a pipeline that can store the data reliably, process it efficiently, and expose it to non-technical users through visual tools.

This report documents Phase 1 of a big data analytics project built around the Olist Brazilian E-Commerce dataset. The goal was to design and implement a working pipeline that takes raw CSV files all the way to an interactive dashboard, using four open-source technologies: Apache Spark for processing, Hadoop HDFS for storage, Apache Hive for SQL access, and Apache Superset for visualization. All components run as Docker containers on a shared network, so the entire stack can be brought up or torn down as a single unit.

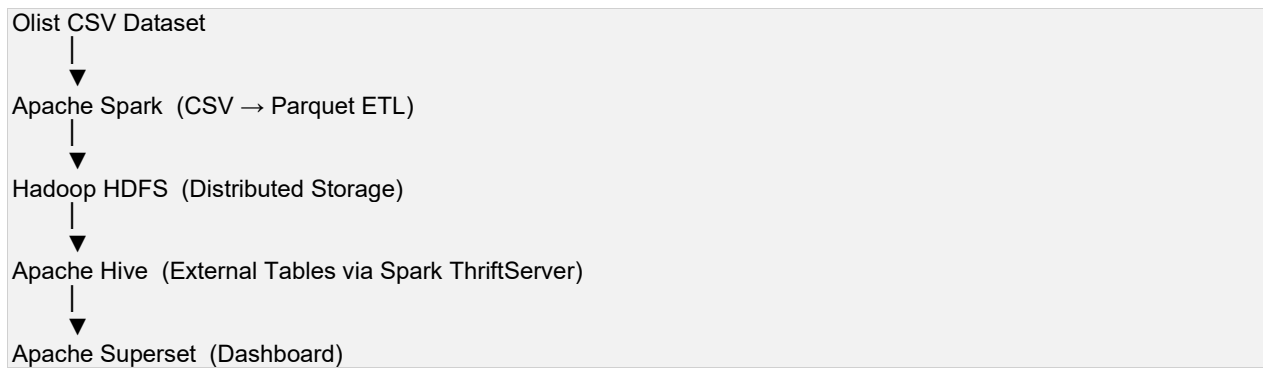
Beyond simply connecting these tools together, this report also covers the practical problems encountered while building the pipeline and how each one was resolved, since debugging a multi-service system is as much a part of the work as the final result.

2. Project Objectives

- Load all 9 raw CSV files of the Olist dataset using Apache Spark, with automatic schema inference.
- Convert every CSV table into Parquet, a compressed, columnar format better suited to analytical queries.
- Persist the Parquet output in Hadoop HDFS rather than on local disk, so storage is distributed and shared across services.
- Expose the HDFS data as SQL-queryable tables via the Hive Metastore, without duplicating the underlying files.
- Connect Apache Superset to that SQL layer and build a dashboard covering the dataset's key business metrics.
- Automate the repetitive setup steps (table and dataset registration) with a script, so the pipeline can be rebuilt without manually repeating UI actions.

3. System Architecture

The pipeline is organized as five layers, each handling one responsibility and passing its output to the next:



Each stage is deliberately decoupled from the others. Spark does not know or care how Superset will query its output; it only needs to write valid Parquet files to a location in HDFS. Hive, in turn, does not store any data itself it only keeps a schema and a pointer to where the data lives, which means re-running the Spark job and overwriting the Parquet files does not require re-registering anything downstream. This separation is what makes the pipeline reproducible: any single layer can be rebuilt independently of the others.

All services communicate over a single Docker network (bigdata-net), with each component running in its own container: an HDFS NameNode and DataNode, a Spark Master with one worker, a Spark ThriftServer acting as the Hive-compatible SQL endpoint, and a Superset instance backed by its own Postgres and Redis containers.

4. Data Processing Pipeline

The source data is the Brazilian E-Commerce Public Dataset published by Olist, covering roughly 100,000 real orders placed between 2016 and 2018, split across 9 CSV files: customers, orders, order items, payments, reviews, products, sellers, geolocation records, and a category name translation table.

4.1 Ingestion and Schema Inference

A single PySpark script (analysis.py) iterates over the 9 CSV files. Each file is read with schema inference enabled, so dates and numeric fields are typed correctly instead of being treated as plain strings this matters later, since several Superset charts group data by month or by hour directly from a timestamp column, which only works if that column was read as an actual timestamp type rather than text.

4.2 Conversion to Parquet

Each DataFrame is written out as Parquet rather than staying in CSV form. Parquet stores data column-by-column and compresses it, which reduces file size and lets query engines skip columns that are not needed for a given query something that is not possible with a row-oriented, uncompressed format like CSV.

4.3 Distributed Storage in HDFS

The Parquet output is written directly to `hdfs://namenode:9000/data/olist/`, with one subdirectory per table. Writing straight to HDFS instead of a local path means the data is immediately available to any other container on the network Spark does not need to be the one to move it anywhere afterward.

4.4 Query Layer via Hive External Tables

HDFS on its own is just a filesystem; nothing about it is queryable with SQL. To bridge that gap, each of the 9 HDFS directories is registered as a Hive external table through the Spark ThriftServer, which exposes a HiveServer2-compatible endpoint on port 10000:

```
CREATE TABLE olist_customers_dataset
USING PARQUET
LOCATION 'hdfs://namenode:9000/data/olist/olist_customers_dataset';
```

Because the table is external, Hive stores only the schema and the HDFS path the Parquet files themselves are never copied. If the ETL step is re-run and the files are overwritten, every table that points at that location immediately reflects the new data with no further action required.

5. Automation and Reproducibility

Registering 9 Hive tables and 9 Superset datasets by hand through the SQL Lab and Datasets screens is repetitive and easy to get wrong, especially after the Hive Metastore is reset (see Section 7). To avoid repeating this manually every time the environment is rebuilt, a helper script, `register_tables.py`, was written to automate both steps.

The script runs in two stages. First, it connects to the Spark ThriftServer using the `pyhive` library and issues a `CREATE TABLE ... USING PARQUET` statement for each of the 9 tables. Second, it authenticates against the Superset REST API obtaining a JWT access token and a CSRF token and creates the Hive database connection and each dataset through the API rather than the UI. Every step checks for an existing record before creating a new one, so the script can be run repeatedly without producing duplicates.

Because the Superset image does not mount the project directory, the script has to be copied into the container before it can run:

```
docker cp visualization/register_tables.py superset:/tmp/register_tables.py
docker exec -it superset python /tmp/register_tables.py
```

6. Dashboard Analysis

The final layer of the pipeline is an Apache Superset dashboard built directly on top of the Hive tables and a small number of derived datasets created with SQL joins in SQL Lab. Figure 1 shows the dashboard as configured at the end of Phase 1.

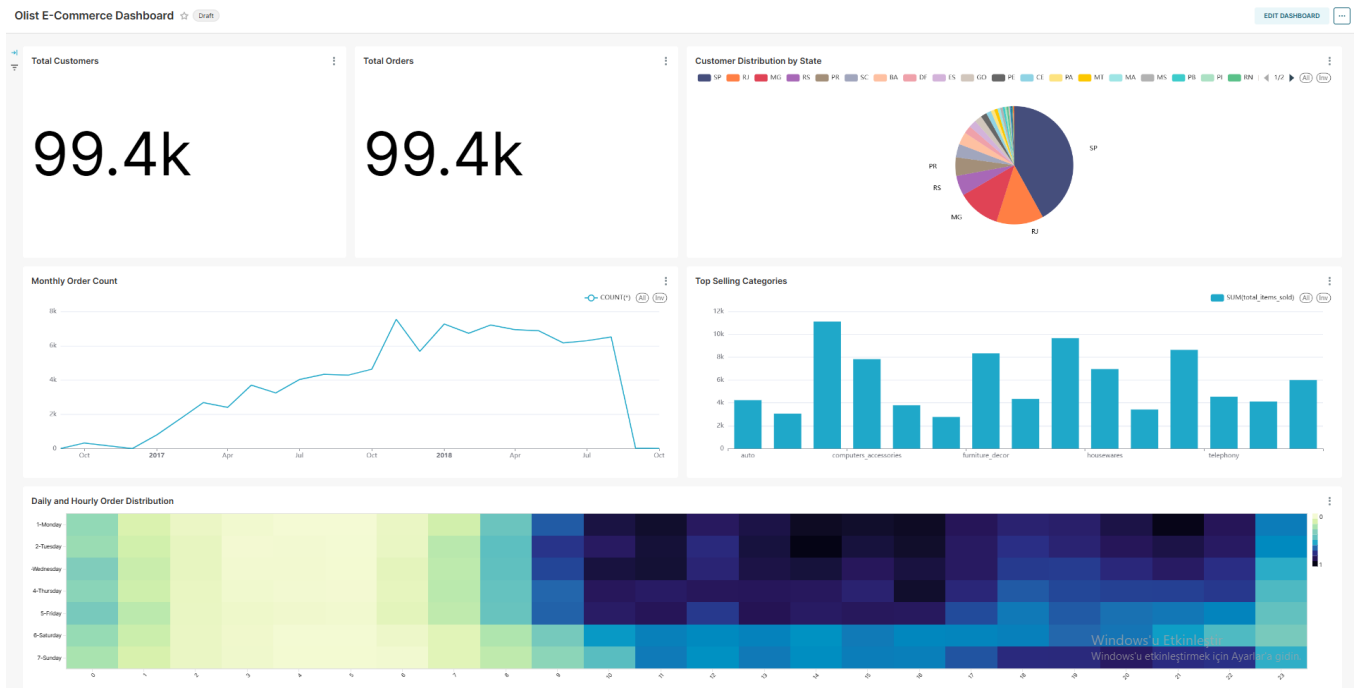


Figure 1: Olist E-Commerce dashboard in Apache Superset.

6.1 Total Orders and Total Customers

Two Big Number charts give an immediate read on the overall scale of the marketplace: the total row count of the orders table and the total row count of the customers table. These act as reference points every other chart in the dashboard is, in effect, a breakdown of one of these two totals along some dimension (time, category, or geography).

6.2 Monthly Order Count

A line chart groups orders by month using `order_purchase_timestamp`, showing order volume from October 2016 through October 2018. The chart makes the platform's growth trajectory visible at a glance: order volume rises sharply through 2017 before leveling off in 2018, which would not be obvious from the single Total Orders number alone.

6.3 Top Selling Categories

Product category is not stored directly on the order items table; it has to be reached by joining order items to products, and products to the category name translation table. A SQL Lab query performs this three-way join, aggregates by category, and ranks categories by items sold, which is then saved as its own dataset (`category_sales_summary`) and rendered as a bar chart.

6.4 Customer Distribution by State

A pie chart groups customers by `customer_state` across all 27 Brazilian states. São Paulo (SP) accounts for the largest share by a wide margin, followed by Rio de Janeiro (RJ) and Minas Gerais (MG) a pattern consistent with the population and economic concentration of Brazil's southeastern region.

6.5 Orders by Day of Week and Hour

A heatmap cross-references the day of the week with the hour of day extracted from the order timestamp, producing a 7-by-24 grid of order counts. This surfaces a pattern that neither the monthly chart nor a simple daily total could show on its own: purchasing activity concentrates in the late morning through early evening on weekdays, with a visible dip during the very early hours across every day of the week.

7. Challenges Encountered and Solutions

A number of practical issues came up while assembling the pipeline. They are documented here because they were not obvious in advance and took real debugging to resolve.

SQL Lab rejected CREATE TABLE statements. By default, Superset's SQL Lab only permits SELECT statements against a connection. Running CREATE TABLE failed with “Only SELECT statements are allowed” until the Allow DML option was enabled on the Hive database connection under Advanced settings.

STORED AS PARQUET could not infer the schema. Hive's traditional STORED AS PARQUET syntax expects an explicit column list and failed with an AnalysisException when no schema was given. Switching to Spark SQL's native USING PARQUET syntax resolved this, since it reads the schema directly from the existing Parquet files at the given location.

Empty table list in the Superset Dataset screen. Even after the Hive tables existed, the table dropdown when creating a new Superset dataset stayed empty, apparently due to how Superset introspects tables over this particular Hive connection. The workaround was to run a SELECT query in SQL Lab and use “create dataset from SQL query,” which saves the dataset successfully even when the dropdown-based flow does not.

Heatmap displayed days in alphabetical order. Since day of week was extracted as a text value, Superset sorted the heatmap's Y-axis alphabetically (Friday, Monday, Saturday, ...) instead of Monday through Sunday. Prefixing each day name with its numeric weekday index (1-Monday, 2-Tuesday, ...) made the alphabetical sort match the intended chronological order.

Hive Metastore state did not survive a container restart. After the host machine was restarted, the Spark ThriftServer came back up with an empty Hive Metastore, and every dashboard chart failed with “Table or view not found.” The underlying Parquet data in HDFS was untouched only the table registrations were lost, since the ThriftServer's embedded metastore is not itself persisted. Re-running

the CREATE TABLE statements (or the register_tables.py script) restored the dashboard without needing to reprocess any data.

register_tables.py could not find its own file inside the Superset container. The Spark containers mount the project directory at /app, but the Superset container defined in docker-compose-superset.yml does not. Adding the same mount to the Superset service risked overwriting Superset's own application code, which also lives under /app inside that image. Instead, the script is copied into the container with docker cp immediately before it is executed, leaving the Superset image itself untouched.

8. Discussion

The clearest lesson from this phase of the project is how much of a distributed pipeline's reliability comes from decisions that are invisible in the final dashboard. Choosing to write Parquet to HDFS instead of local disk, registering Hive tables as external rather than managed, and enabling schema inference on the Spark read were all small decisions, but each one avoided a class of problem that would otherwise have surfaced later: data locked to a single container, having to re-copy files after every ETL run, or charts breaking because a date column was silently read as a string.

It is also worth noting what this phase deliberately did not do: no data cleaning, deduplication, or transformation logic was applied beyond the CSV-to-Parquet conversion itself, in line with the Phase 1 requirement that no advanced transformations were needed. The dashboard therefore reflects the raw dataset as published, which is sufficient for descriptive analytics but would need a cleaning step before being used for anything more rigorous, such as forecasting or customer segmentation.

The automation script also changed how the later stages of the project were approached. Once table and dataset registration could be re-run with a single command, restarting the whole environment after a machine reboot stopped being a source of anxiety and became a two-minute recovery step, which is a meaningful difference for a project developed incrementally over several sessions rather than in one sitting.

9. Conclusion

This phase implemented a complete pipeline from raw CSV files to an interactive dashboard, using Apache Spark for transformation, Hadoop HDFS for distributed storage, Apache Hive for SQL access, and Apache Superset for visualization, with every service running in Docker on a shared network. The resulting dashboard covers order volume over time, category-level sales broken down by month, customer distribution across Brazilian states, and purchasing patterns by day and hour.

Beyond the pipeline itself, documenting the failures along the way: permission errors in SQL Lab, schema inference issues, a metastore that did not survive a restart: turned out to be as useful as the final architecture diagram, since those are the parts most likely to trip up anyone rebuilding this environment from scratch. Future work could extend the pipeline with data quality checks before the Parquet write, a persistent (non-embedded) Hive Metastore so that table registrations survive a restart without re-running a script, and additional dashboard metrics such as delivery time or review score analysis.

References

- [1] Apache Spark. <https://spark.apache.org>
- [2] Apache Hadoop. <https://hadoop.apache.org>
- [3] Apache Hive. <https://hive.apache.org>
- [4] Apache Superset. <https://superset.apache.org>
- [5] Olist Brazilian E-Commerce Dataset. <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Ahmet Berat Yıldırım